

System Design Document

ICU 護理分配決策支援系統

1. 系統概述

本系統協助 ICU 小組長在每班班前，依據病患麻煩度評估結果、病床鄰近性、護理師班表與突發 STAT 醫囑資訊，快速完成護理師分床。分床確認後，系統會產生交班快照，並在班中提供戰情室供小組長追蹤護理師負荷、待辦任務與未完成 STAT 醫囑。

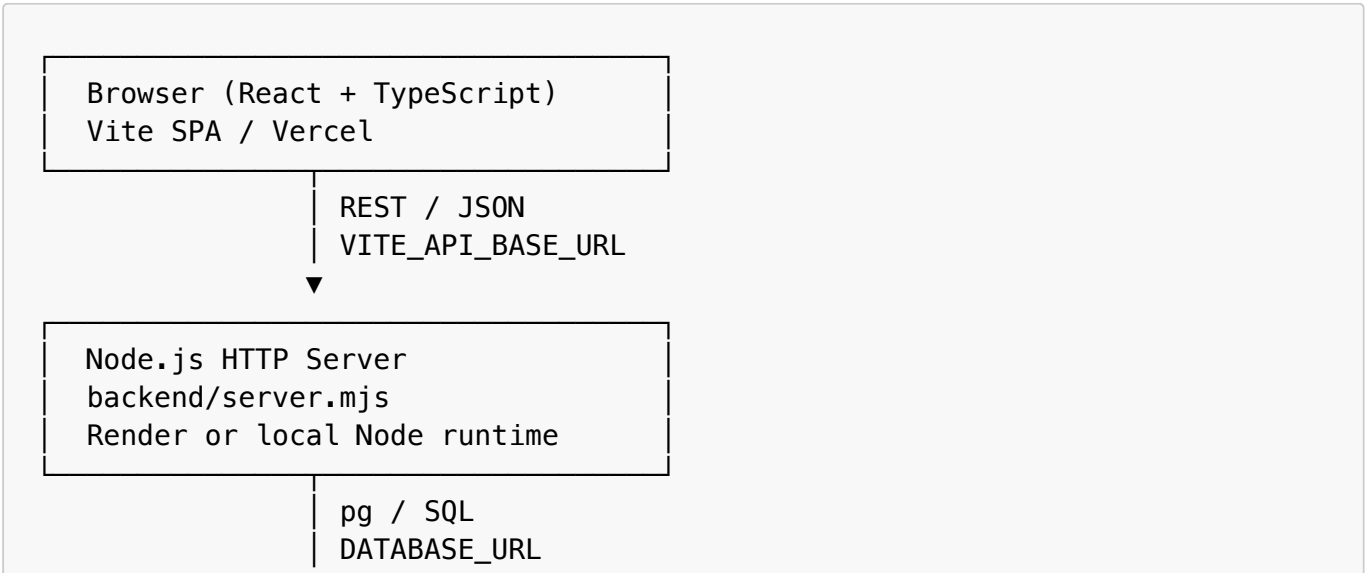
主要目標

目標	說明
降低分床時間	由系統根據病患負荷與床位位置產生分床建議，小組長可再人工調整
提升負荷透明度	以麻煩度分數、任務與 STAT 醫囑呈現各護理師負荷
支援交班紀錄	分床確認後封存交班快照，保留當下分床與病患摘要
支援班表匯入	由 xlsx 班表資料產生各班出勤護理師與小組長資料

主要使用者角色

角色	說明
護理師 (nurse)	查看本班病患、填寫病患麻煩度評估、查看與完成待辦任務
小組長 / 小組長 (charge_nurse)	產生分床建議、拖曳調整分配、確認分床、查看戰情室與交班快照
醫師 / 醫囑輸入者	新增 STAT 突發醫囑，供護理師與小組長即時查看
管理員 (admin)	系統資料維護與管理；目前主要保留於角色模型中

2. 系統架構



PostgreSQL
Schema migrations + demo seeds

技術選型

層級	技術
前端框架	React 19 + TypeScript
前端建置	Vite
前端路由	react-router-dom
前端資料請求	fetch wrapper (<code>src/api/client.ts</code>)；部分頁面使用 TanStack Query
拖曳互動	<code>@dnd-kit/core</code> 、 <code>@dnd-kit/sortable</code>
班表匯入	<code>xlsx</code>
後端	Node.js 內建 <code>http</code> module（無 Express 等框架）
資料庫	PostgreSQL + <code>pg</code> client
部署（前端）	Vercel
部署（後端）	Render 或本機 Node process

3. 前端頁面結構

頁面檔案	路由	功能
<code>NurseOverviewPage.tsx</code>	<code>/nurse/overview</code>	護理師查看本班病患總覽
<code>BurdenFormPage.tsx</code>	<code>/nurse/burden-form</code>	填寫病患麻煩度評估表，送出 <code>draft</code> / <code>submitted</code> 狀態
<code>NurseTodoPage.tsx</code>	<code>/nurse/todo</code>	查看一般任務與個人 STAT 醫囑，並更新完成狀態
<code>ChargeAllocationPage.tsx</code>	<code>/leader/allocation</code>	小組長產生分床建議、拖曳調整、查看決策紀錄、確認分床
<code>AllocationResultPage.tsx</code>	<code>/leader/allocation-result</code>	查看已確認分床後產生的交班單
<code>WarRoomPage.tsx</code>	<code>/leader/war-room</code>	即時戰情室，依已確認分床彙整各護理師病患、任務與剩餘負荷
<code>HandoverSnapshotsPage.tsx</code>	<code>/leader/handoff</code>	查看歷史交班快照與分床封存內容
<code>RosterImportPage.tsx</code>	<code>/roster-import</code>	匯入護理師班表 <code>xlsx</code> ，建立班別與出勤護理師

頁面檔案	路由	功能
DoctorStatPage.tsx	/doctor/stat	新增或匯入 STAT 突發醫囑

前端 API 客戶端：`src/api/client.ts`，預設指向 `http://127.0.0.1:8787/api/v1`，可透過環境變數 `VITE_API_BASE_URL` 覆蓋。

4. 後端 API 設計

Base URL：`/api/v1`

通用回應格式

```
// 成功
{ "data": {}, "meta": {} }

// 錯誤
{ "error": { "code": "VALIDATION_ERROR", "message": "...", "details": {} } }
```

目前 API 使用 `X-User-Id` header 或 `userId` query parameter 模擬目前使用者。若未指定，系統使用 demo user。正式上線時應改為登入 session 或 token 驗證。

端點一覽

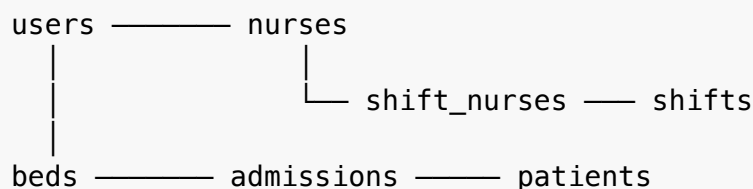
方法	路由	說明
GET	/health	服務健康檢查
GET	/me	取得目前使用者
GET	/shifts	取得班別清單
GET	/shifts/current	取得目前開放中的 ICU 班別
GET	/nurses	取得指定班別的出勤護理師
GET	/admissions	取得住院病患清單，可依 <code>status</code> 篩選
GET	/nurse/overview	護理師病患負荷總覽
GET	/burden-assessments	取得麻煩度評估列表，可用 <code>scope=mine</code> 篩選個人
PATCH	/burden-assessments/{assessmentId}	更新麻煩度評估與狀態
GET	/tasks	取得一般待辦任務
PATCH	/tasks/{taskId}	更新一般任務狀態
GET	/stat-orders	取得 STAT 醫囑，可篩選 assignee 與是否包含完成項目

方法	路由	說明
POST	<code>/stat-orders</code>	新增 STAT 醫囑
POST	<code>/stat-orders/import</code>	匯入 demo STAT 醫囑
PATCH	<code>/stat-orders/{orderId}</code>	更新 STAT 醫囑狀態
POST	<code>/roster/import</code>	匯入班表資料，建立 shifts 與 shift_nurses
GET	<code>/allocation-runs/current</code>	取得指定班別最新分床執行結果
POST	<code>/allocation-runs/suggest</code>	產生系統分床建議，可支援 dry-run 決策紀錄
GET	<code>/allocation-runs/{allocationRunId}</code>	取得分床結果
PUT	<code>/allocation-runs/{allocationRunId}/items</code>	更新分床項目（人工調整）
POST	<code>/allocation-runs/{allocationRunId}/confirm</code>	確認分床；同時建立交班快照
POST	<code>/allocation-runs/{allocationRunId}/revert-to-draft</code>	將已確認分床退回草稿，並刪除對應交班快照
GET	<code>/war-room</code>	取得戰情室資料
GET	<code>/handoff-sheets</code>	取得指定班別最新交班單
GET	<code>/handoff-snapshots</code>	取得交班快照清單
GET	<code>/handoff-snapshots/{snapshotId}</code>	取得單一交班快照內容

5. 資料庫設計

系統使用 PostgreSQL，schema 由 `backend/db/migrations/` 管理，demo 資料由 `backend/db/seeds/` 與 setup script 匯入。

5.1 核心領域 (Migration 001)



資料表	說明
<code>users</code>	系統使用者，role 包含 <code>nurse</code> 、 <code>charge_nurse</code> 、 <code>admin</code>
<code>nurses</code>	護理師擴充資料，包含顯示名稱、簡稱、年資等級與啟用狀態
<code>shifts</code>	班別資料，包含 ICU 單位、day / evening / night、起訖時間、小組長與班別狀態

資料表	說明
<code>shift_nurses</code>	班別與護理師的多對多關係，記錄當班角色
<code>beds</code>	ICU 病床主檔
<code>patients</code>	病患基本資料
<code>admissions</code>	住院記錄，連結病患與病床，記錄診斷、主治醫師與住院狀態

重要限制：

- `admissions_one_active_per_bed` 確保同一病床同時間只能有一筆 active admission。
- `shift_nurses` 以 (`shift_id`, `nurse_id`) 作為 primary key，避免同一護理師在同一班重複出現。

5.2 麻煩度評估與一般任務 (Migration 002)

```

burden_factors
├── burden_values — burden_assessments — shifts
│                                   │
│                                   └── admissions
tasks — shifts / admissions / nurses

```

資料表	說明
<code>burden_factors</code>	麻煩度因素定義，分為 objective / subjective，支援 number / boolean / level
<code>burden_assessments</code>	每班每位病患的評估主檔，包含 objective_total、subjective_total、total_score 與 draft / submitted 狀態
<code>burden_values</code>	各評估因素的實際填寫值與對應點數
<code>tasks</code>	一般護理待辦任務，包含負責護理師、急件旗標、來源與完成狀態

主觀因素計分規則

類型	規則
RASS 分數	絕對值 0-1 為 0 分、2-3 為 1 分、4 以上為 2 分
Boolean 因素	true 為 2 分、false 為 0 分
Level 因素	low / 0 為 0 分，medium / 1 為 1 分，high / 2 為 2 分

5.3 分床 (Migration 003 + 006)

```

allocation_runs — shifts
├── allocation_items — admissions

```

```

└─ nurses

```

資料表	說明
<code>allocation_runs</code>	一次分床執行記錄，包含來源班別、目標班別、建立者、狀態、演算法版本、建議時間、確認時間
<code>allocation_items</code>	分床項目，每位病患對應一位護理師，保存分數、排序與是否人工調整
<code>allocation_runs.decision_logs</code>	Migration 006 新增的 JSONB 欄位，保存系統分床時每位病患的候選護理師、階段、是否高負荷與最終選擇

分床狀態：

- `draft`：小組長可拖曳調整。
- `confirmed`：結果已確認，不可再直接修改；系統會產生交班快照。
- `cancelled`：保留於資料模型，目前前端主要使用 `draft` / `confirmed`。

5.4 STAT 醫囑 (Migration 004)

資料表	說明
<code>stat_orders</code>	突發 STAT 醫囑，連結 shift 與 admission，包含標題、類型、醫囑來源、時間顯示、原因與狀態

STAT 醫囑狀態包含 `pending`、`done`、`cancelled`。戰情室與護理師待辦頁會將 STAT 醫囑合併進任務視圖，讓突發工作可即時反映在護理師負荷上。

5.5 交班快照 (Migration 005)

```

handoff_snapshots — allocation_runs
└─ handoff_rows — admissions

```

資料表	說明
<code>handoff_snapshots</code>	分床確認時建立的交班封存主檔，包含病患數、護理師數、STAT 總數、平均負荷、最高負荷、未分配數與 <code>nurse_blocks</code> JSON
<code>handoff_rows</code>	交班明細，每列對應一位病患，保存床號、姓名、診斷、主治醫師、目前護理師、下一班護理師與麻煩度摘要

設計目的：

- 分床確認後，即使後續任務、病患或評估資料變動，交班快照仍保留確認當下的分床與摘要。
- `allocation_run_id` 在 `handoff_snapshots` 中為 `unique`，避免同一次分床重複產生多份交班快照。

6. 分床演算法

目前版本標識：`bed-proximity-v4`

分床演算法的目標不是只讓總分平均，也要盡量讓同一位護理師負責的床位相近，降低 ICU 班中移動與交接成本。

主要輸入

輸入	說明
班別病患	來源班別 <code>shiftId</code> 的 active admissions、床號、診斷與病患基本資料
麻煩度分數	以 <code>burden_assessments.total_score</code> 作為主要負荷
出勤護理師	由目標班別 <code>targetShiftId</code> 的 <code>shift_nurses</code> 與 <code>nurses</code> 取得，包含角色與年資
目標班別	<code>targetShiftId</code> 可用於替下一班產生分床；未指定時系統會嘗試取下一班，最後才 fallback 回來源班別

床位距離規則

ICU 床位以走廊順序 1–17 建模，並視為環狀動線，因此 17 號床與 1 號床相鄰。兩床距離使用 `walkDist` 計算順時針與逆時針的較短距離；同一護理師負責的床位組會用 `maxPairWalkDist` 檢查任兩床最大距離。系統目前設定 `MAX_BED_GAP = 2`，代表同一組床位盡量維持在走廊距離 2 以內。

主要流程

- 配額計算：**依病患數與出勤護理師數計算每位護理師的基本床數，餘數由排序較前的護理師多分一床。
- 床位分組：**依床號走廊順序掃描病患，貪婪形成不超過 `MAX_BED_GAP` 的床位組。
- 鄰組交換：**在相鄰床位組之間嘗試單一病患交換，讓每組盡量同時包含高於平均負荷與低於平均負荷的病患；交換後仍必須符合床位距離限制。
- 組別對應護理師：**計算各組總負荷後，小組長優先取得最輕組；其餘組別依負荷由輕到重，分配給年資由深到淺的一般護理師。
- 溢出處理：**若有病患因床位距離限制未被分入配額組，系統優先指派給已有鄰近床位的護理師；若沒有鄰近候選者，則指派給目前累計負荷最低者。
- 保存結果：**將每位病患的候選護理師、決策階段、分數與最終選擇寫入 `allocation_runs.decision_logs`，並建立 `allocation_runs` 與 `allocation_items`，狀態為 `draft`。

人工調整與確認

操作	行為
拖曳調整	前端更新分床版面，呼叫 <code>PUT /allocation-runs/{id}/items</code> 保存草稿
人工調整標記	被人工調整的項目以 <code>is_manual_override = true</code> 保存
確認分床	必須沒有未分配病患；確認後 <code>status = confirmed</code> 並寫入 <code>confirmed_at</code>
產生交班	確認分床時自動建立 <code>handoff_snapshots</code> 與 <code>handoff_rows</code>

操作	行為
退回草稿	<code>revert-to-draft</code> 會清除 <code>confirmed_at</code> 並刪除該 run 的交班快照

7. 戰情室與交班設計

7.1 戰情室

戰情室使用最新已確認的分床結果，將病患與任務重新彙整到各護理師底下。資料來源包含：

來源	用途
<code>allocation_runs / allocation_items</code>	判斷每位病患目前由哪位護理師負責
<code>tasks</code>	一般待辦任務
<code>stat_orders</code>	突發 STAT 醫囑，轉成任務樣式顯示
<code>burden_assessments</code>	病患麻煩度分數與負荷分級

戰情室回傳每位護理師的病患清單、任務清單、剩餘任務點數、病患負荷與總覽統計。

7.2 交班快照

交班資料不是每次即時計算，而是在分床確認時封存。這樣可以保證交班單代表「確認分床當下」的正式版本。

交班快照包含：

- 班別、建立時間、建立者。
- 分床摘要：病患數、護理師數、STAT 數量、平均負荷、最高負荷、未分配數。
- 各護理師負責床位與病患負荷。
- 每位病患的床號、姓名、診斷、主治醫師、目前護理師、下一班護理師與麻煩度細節。

8. 環境變數

變數	預設值	說明
<code>HOST</code>	<code>127.0.0.1</code>	後端監聽 host
<code>PORT</code>	<code>8787</code>	後端監聽 port
<code>DATABASE_URL</code>	<code>postgresql://postgres@%2Ftmp/sad_frontend_v2</code>	應用程式 DB 連線
<code>DATABASE_ADMIN_URL</code>	<code>postgresql://postgres@%2Ftmp/postgres</code>	DB 建立 / 重置用連線
<code>DATABASE_SETUP_MODE</code>	未設定	設為 <code>schema</code> 時可用於雲端環境 schema setup

變數	預設值	說明
VITE_API_BASE_URL	http://127.0.0.1:8787/api/v1	前端 API 目標

9. 後端模組結構

檔案	說明
backend/server.mjs	HTTP server 進入點，處理 CORS、路由分發、JSON body 與錯誤回應
backend/src/runtimeRepository.mjs	API 使用的 repository 轉接點，目前指向 PostgreSQL repository
backend/src/pgRepository.mjs	PostgreSQL 查詢與主要業務邏輯：評估、任務、STAT、分床、戰情室、交班
backend/src/db.mjs	pg 連線池
backend/scripts/setupDatabase.mjs	建立 DB、套用 migration、植入 seed 與 demo 資料
backend/scripts/seedSixShiftAllocations.mjs	產生多班別分床 demo 資料
backend/db/migrations/	Schema migration SQL (001-006)
backend/db/seeds/	Demo seed SQL (001-004)

10. 資料流程

10.1 麻煩度評估流程

1. 護理師進入 `/nurse/burden-form`。
2. 前端呼叫 `/burden-assessments?shiftId=...&scope=mine` 取得本班評估資料。
3. 護理師填寫主觀因素並儲存 draft 或 submitted。
4. 後端重新計算 subjective_total 與 total_score。
5. 分床演算法後續以 total_score 作為主要負荷依據。

10.2 分床與交班流程

1. 小組長進入 `/leader/allocation`。
2. 前端讀取護理師、住院病患、目前分床草稿、麻煩度與 STAT 醫囑。
3. 小組長呼叫 `/allocation-runs/suggest` 產生分床建議。
4. 小組長可拖曳調整分配，前端呼叫 `/allocation-runs/{id}/items` 保存草稿。
5. 小組長確認分床，後端檢查是否仍有未分配病患。
6. 若檢查通過，分床狀態改為 confirmed，並建立交班快照。
7. `/leader/allocation-result` 與 `/leader/handoff` 顯示封存後的交班資料。

10.3 STAT 醫囑流程

1. 醫師或 demo 使用者在 `/doctor/stat` 建立 STAT 醫囑。

2. 醫囑寫入 `stat_orders`，狀態為 `pending`。
3. 護理師待辦頁與小組長戰情室讀取 STAT 醫囑並合併到任務列表。
4. 使用者可將 STAT 醫囑更新為 `done` 或 `cancelled`。

11. 錯誤處理與設計邊界

錯誤處理

情境	回應
JSON 格式錯誤	400 <code>INVALID_JSON</code>
缺少必要參數	400 <code>MISSING_PARAM</code> 或 400 <code>VALIDATION_ERROR</code>
找不到資源	404 <code>NOT_FOUND</code> 或特定錯誤碼
HTTP method 不支援	405 <code>METHOD_NOT_ALLOWED</code>
已確認分床仍嘗試修改	409 <code>ALLOCATION_RUN_LOCKED</code>
分床仍有未分配病患卻確認	409 <code>ALLOCATION_INCOMPLETE</code>

設計邊界與後續擴充

項目	說明
身份驗證	目前以 <code>X-User-Id</code> 或 query parameter 模擬目前使用者；正式環境可擴充為 session、JWT 或院內 SSO
即時更新	戰情室目前透過前端重新請求 API 取得最新狀態；後續可加入 WebSocket 或 server-sent events 支援推播
分床最佳化	目前採 heuristic 分床，重點是可解釋、可快速產生建議；若 ICU 規模擴大，可評估導入最佳化求解器
醫療系統整合	病患、醫囑、班表目前以 demo 或匯入資料為主；後續可串接 HIS / EMR / 排班系統
稽核紀錄	分床已保存 decision logs；後續可擴充一般任務與 STAT 狀態異動的完整 audit log

12. 建置與啟動

```
npm install
npm run db:setup
npm run api:dev
npm run dev
```

常用指令：

指令	說明
<code>npm run dev</code>	啟動 Vite 前端
<code>npm run api:dev</code>	啟動 Node API server
<code>npm run db:setup</code>	重建本機 PostgreSQL demo database 並套用 seeds
<code>npm run build</code>	TypeScript build + Vite production build
<code>npm run lint</code>	執行 ESLint
<code>npm run preview</code>	預覽 production build

13. 設計亮點

13.1 量化主觀麻煩度

傳統分床依賴護理師主觀經驗，難以客觀比較。本系統將麻煩度拆分為：

- **客觀因素**：由醫囑系統自動帶入（RASS 分數、用藥複雜度、檢查項目等）
- **主觀因素**：護理師依臨床觀察填寫（照護難度、溝通困難度等）

兩者合計為統一分數，作為分床演算法的主要依據，讓負荷分配有量化基礎。

13.2 圖形化拖曳分床

小組長可在視覺化看板上直接拖曳病患至不同護理師欄位，系統即時更新負荷數值與顏色標示，降低調整成本。突發病患轉入時，小組長可立即在看板上指派，不需重新計算整體分配。

13.3 小組長戰情室

確認分床後，戰情室依最新分配即時彙整：

- 每位護理師的負責病患清單
- 一般任務與 STAT 醫囑的完成進度
- 剩餘任務點數與負荷分級（視覺化顏色標示）

讓小組長在不打擾護理師工作的情況下掌握全局。

14. 非功能需求

需求	系統設計
易用性	介面以臨床使用習慣設計，拖曳操作直覺；關鍵動作（確認分床、核銷任務）有明確視覺反饋
即時性	麻煩度填寫即時計算總分；建議分床於後端即時運算，前端無需等待重整
協作性	多護理師可同時填寫評估，互不衝突；小組長即時看到最新填寫狀態
提示機制	有未分配病患時封鎖確認；麻煩度未完整填寫時顯示警告

需求

系統設計

流程可見性

分床狀態（草稿 / 確認）、任務狀態（待辦 / 完成）均在介面清楚呈現